

Individual Project Interim Report: Big-Picture Visualisation

Tania Pan Group 4

1. Introduction and Motivation

The group project (GP) architecture of the Extended Automated Bottling System (EABS) consists of seven controller/plant pairs coordinated by a System Coordinator, feeding a Digital Twin and Big-Picture Visualisation module as shown in Figure 1 of the GP report. This gives an operator an overview of the whole line, but a single system-level view is, by design, too coarse to answer many questions at a glance: which stage are individual bottles currently at, where has a fault occurred, or which batch is currently being made and what is its progress?

This individual project (IP) generalises that experience into a single interactive visualisation tool that shows the whole EABS at the system level by default, and lets a user click into any individual machine to see a detailed, machine-specific view, eliminating the need to have every machine's GUI open to check the progress of a batch.

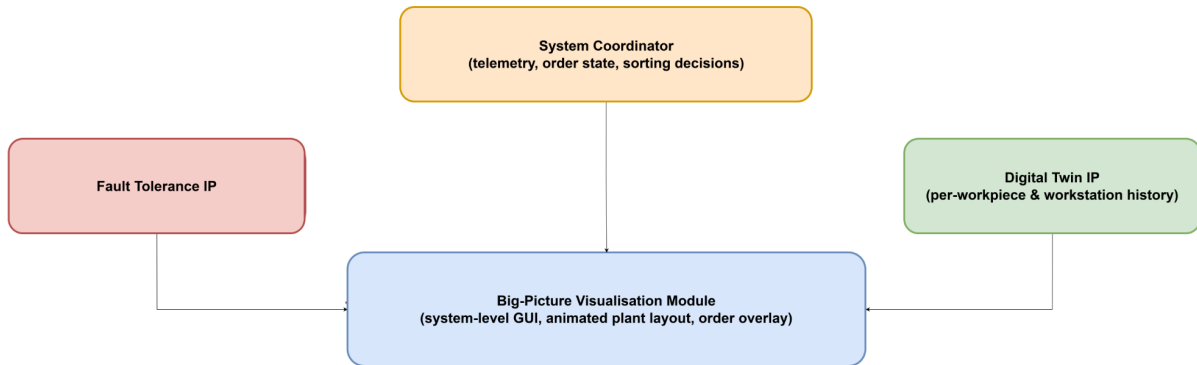
2. IP Goals

- Provide a single Big-Picture view of the EABS showing every machine's high-level status (idle, active, faulted) and the position of bottles currently in the system.
- Allow a user to click on any machine in the Big-Picture view and open a detailed panel for that machine, showing its controller state, plant state, and the live values of its interface signals.
- Define a single, generic interface between a machine's controller/plant model and the visualisation layer, so that adding a new machine to the EABS does not require writing a new bespoke GUI.
- Present fault state and digital-twin history (from Chris and Marie's IPs) within the same GUI, so a user does not need to switch between separate tools to see a machine's current status, its fault history, and its twin record.

3. What the Visualisation Tracks and Displays

A diagram of the information feeding the Big-Picture Visualisation can be found below in Panel A of Figure 1. A rough layout mockup of the visualisation can be found in panel B of Figure 1. The mockup has been simplified for now due to not knowing what complexity of graphics can be achieved during implementation, but the end goal of this IP is to create a much more detailed and complex design than what is pictured in Figure 1.

A. Data feeding the Big-Picture Visualisation



B. Layout mockup

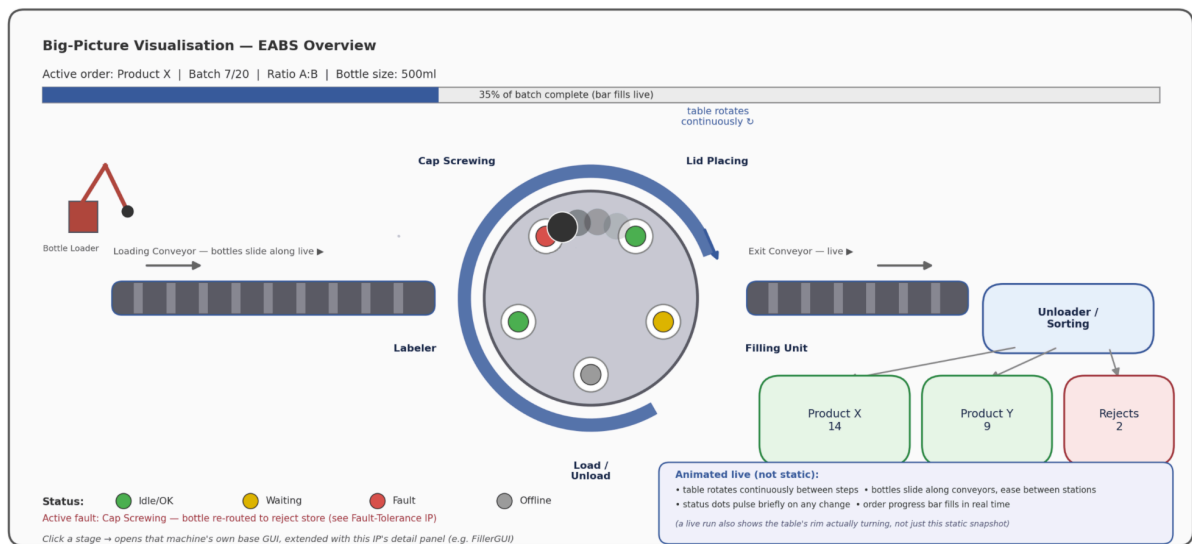


Figure 1. Mockup of the Big-Picture GUI.

The module tracks and renders the following pieces of status, each sourced from a signal or module that already exists elsewhere in the group's architecture rather than a new sensor:

Tracked status	Source	Displayed as
Active order / batch identity	POS → System Coordinator	Header text: product, batch count, liquid ratio, bottle size
Batch progress	System Coordinator telemetry	Progress bar (bottles completed / batch size)
Per-stage machine state (idle / active / waiting / faulted / offline)	Each controller's status signal, already reported to the System Coordinator	Colour-coded status dot on each of the 7 pipeline stage icons
Workpiece position	atPos1—atPos6 / xDone handshake signals between controllers	A marker that moves along the pipeline as the bottle advances

Active faults and mitigation action	Fault-Tolerance IP	Red banner naming the affected stage and the action taken (e.g. re-route to reject store)
Digital twin summary (recipe, timestamps, fault history)	Digital-Twin IP, keyed by the product ID applied at the Labeler	Pop-up panel when a workpiece marker is clicked
Storage tally	Unloader/Sorting Controller via System Coordinator	Running counts per product and per reject bin
Machine detail (valve state, gripper position, plus recent event history, fault/twin context)	That machine's base GUI, extended by this IP with an added detail panel	Base GUI opened when a stage icon is clicked, with the extra panel this IP contributes alongside it

4. Methodology

The aggregation layer is a passive observer. It subscribes to the same status channel the System Coordinator already uses to collect telemetry from each controller, plus the fault and twin summaries published by the other two IPs, and will aim to look like the visualisation shown in Panel B of Figure 1. It never emits control signals back into the pipeline, so it cannot affect the timing or correctness of the production line it displays. The IP will aim to animate the turntable, so when the bottles need to move to the next stage of the production line, the visualisation will reflect this. The conveyor belt and workpieces should also be animated to show the positioning of the workpieces.

The extension layer adds a shared detail panel to every machine's base GUI. Rather than writing a bespoke addition to each of the seven GUIs individually, a single panel format will be defined. The fields it shows and how it is populated from the shared telemetry channel and each machine's own developer adds one slot in their GUI for it to attach to. This keeps the panel consistent across machines built by three different people, and means adding a new machine to the pipeline later only requires wiring up one slot rather than a custom integration. The pipeline overview itself will be a layout of seven stage widgets in production order, each bound to that stage's status field from the Coordinator's telemetry stream, and workpiece markers will be positioned using the same atPosN presence signals already defined for each stage.

5. Build Plan

1. Define the Coordinator telemetry contract: agree with the group on a single status message per machine (stage name, state enum, current workpiece ID if any) that every controller reports to the System Coordinator, since every later step depends on this being consistent across all seven machines.
2. Define the shared detail-panel format that every base GUI will host including which fields it shows and how it is triggered, and agree on the slot each machine's GUI needs to expose, so the three of us build compatible, not competing, interfaces.
3. Prototype the pipeline overview and the first detail-panel extension using the Filling Unit (the machine that was finished for the interim) as the only live stage, reusing FillerGUT's existing event feed, to validate both layers end to end before the other six machines are ready.
4. Add the remaining six stage widgets and their detail-panel extensions as their controller/plant models are completed by the rest of the group, keeping each one bound only to the shared telemetry contract and panel format from steps 1 and 2.

5. Implement workpiece markers along the pipeline using the atPosN signals, and the batch/order header using the parameters the System Coordinator receives from the POS.
6. Integrate the fault banner once the Fault-Tolerance IP publishes a fault/mitigation summary on the shared channel, and the digital-twin pop-up once the Digital-Twin IP exposes a per-product-ID lookup.
7. Run a full simulated purchase order end to end and confirm the overview, and every extended machine panel stays consistent with each machine's own base GUI throughout, including during an injected fault.

6. Integration with the Group Project

This IP is the Big-Picture Visualisation shown in the System Overview of the GP report, so it is one of the group's core visible outputs rather than an add-on, and its extension layer touches every other machine GUI in the pipeline. It depends on, but does not duplicate, three things the rest of the group already owns: the System Coordinator's telemetry aggregation, each member's own base per-machine GUI, and the fault and twin summaries produced by the other two IPs. Agreeing the shared telemetry contract and detail-panel format in the first two steps of the build plan is the main coordination point with the rest of the group and is the first thing we will confirm together after the Milestone 1 interview.

7. Milestone 2 Plan

Following the build plan in Section 6, by Milestone 2 the Big-Picture Visualisation will show all seven pipeline stages live, with workpiece tracking, batch progress, the fault banner, digital-twin pop-ups, and a working detail-panel extension on every machine's base GUI. We will validate it during the group's end-to-end demonstration of a full purchase order, including at least one deliberately injected fault, to show the overview and every extended machine GUI remain consistent with each other throughout.